Tutorial – 4

Code

```
"""
Tutorial 4: Search Engine Ranking and Evaluation
================================================
Application : Mini search engine for a college library.
Objective   : Rank library documents for a student's query using
              - Vector Space Model (cosine similarity on TF-IDF vectors)
              - TF-IDF scoring
              - Binary Independence Model (BIM)
              - Probabilistic Retrieval Model (Okapi BM25)
              and evaluate them with Precision, Recall, F-measure,
              MRR, MAP and NDCG using relevance judgments.
"""

import os
import re
import math
from collections import Counter
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import nltk
from nltk.stem import PorterStemmer

# ======================================================================
#  1. Document Collection (college library: titles + descriptions)
# ======================================================================
DOCUMENTS = {
    "D1":  ("Machine Learning in Medical Diagnosis",
            "This book explains how machine learning algorithms such as decision "
            "trees and support vector machines help doctors in medical diagnosis, "
            "with case studies from hospitals and healthcare clinics."),
    "D2":  ("Healthcare Data Analysis using AI",
            "An introduction to artificial intelligence and data analytics for "
            "healthcare records, patient monitoring and hospital management."),
    "D3":  ("History of Computer Networks",
            "Traces the growth of computer networks from ARPANET to the modern "
            "internet and their applications in business and education."),
    "D4":  ("Deep Learning for Disease Prediction",
            "Research paper abstract: deep learning models, a branch of machine "
            "learning, use neural networks to predict diabetes and heart disease "
            "from patient health records."),
    "D5":  ("Database Management Systems",
            "Covers relational databases, SQL, transactions and indexing, with "
            "applications in banking and hospital record keeping."),
    "D6":  ("Machine Learning: Theory and Algorithms",
            "A textbook on machine learning theory: supervised learning, "
            "unsupervised learning, learning theory, regression and machine "
            "learning optimisation algorithms."),
    "D7":  ("Applications of Machine Learning in Healthcare",
            "A survey of machine learning applications in healthcare, including "
            "diagnosis, drug discovery, patient risk prediction and personalised "
            "treatment in healthcare systems."),
    "D8":  ("Public Health and Healthcare Policy",
            "Discusses healthcare policy, public health programmes, insurance "
            "and the economics of healthcare delivery in developing countries."),
    "D9":  ("Machine Learning Applications in Finance and Marketing",
            "Explores machine learning applications for stock prediction, fraud "
            "detection, customer segmentation and marketing analytics."),
    "D10": ("Natural Language Processing of Clinical Notes",
            "Research paper abstract: machine learning models extract diagnoses "
            "and medications from clinical notes in electronic healthcare records."),
    "D11": ("Computer Vision for Medical Imaging",
            "Deep neural networks detect tumours in X-ray and MRI scans, "
            "supporting radiologists in hospitals with faster diagnosis."),
    "D12": ("Operating System Concepts",
            "Covers processes, threads, memory management, file systems and "
            "scheduling algorithms in modern operating systems."),
    "D13": ("Reinforcement Learning for Robotics",
            "Robots learn walking and grasping through reinforcement learning, "
            "reward design and simulation-to-reality transfer."),
    "D14": ("Organic Chemistry Fundamentals",
            "Covers hydrocarbons, functional groups, reaction mechanisms and "
            "laboratory synthesis techniques."),
    "D15": ("Thermodynamics for Engineers",
```

```
            "Laws of thermodynamics, entropy, heat engines and refrigeration "
            "cycles with solved engineering problems."),
    "D16": ("Principles of Microeconomics",
            "Demand and supply, market structures, consumer behaviour and "
            "the economics of public goods."),
    "D17": ("Data Structures and Algorithms",
            "Arrays, linked lists, trees, graphs, sorting and searching "
            "algorithms with complexity analysis."),
    "D18": ("Medieval Indian History",
            "Political and cultural history of the Delhi Sultanate and the "
            "Mughal empire."),
    "D19": ("Wearable Devices for Patient Monitoring",
            "Sensors and wearable devices record heart rate and blood pressure "
            "for remote patient monitoring by doctors."),
    "D20": ("Hospital Information Systems",
            "Design of hospital information systems: patient records "
            "databases, billing, laboratory and pharmacy modules."),
}

# ======================================================================
#  2. Queries and Relevance Judgments (marked by a group of students)
#     Grades: 2 = highly relevant, 1 = partially relevant, 0 = non-relevant
#     Documents not listed are non-relevant (grade 0).
# ======================================================================
QUERIES = {
    "Q1": "machine learning applications in healthcare",
    "Q2": "deep learning for disease prediction",
    "Q3": "database systems for hospital records",
}

JUDGMENTS = {
    "Q1": {"D7": 2, "D1": 2, "D4": 2, "D10": 2, "D2": 1, "D11": 1},
    "Q2": {"D4": 2, "D11": 1, "D7": 1, "D1": 1},
    "Q3": {"D20": 2, "D5": 2, "D2": 1, "D10": 1},
}

MAIN_QUERY = "Q1"
TOP_K = 5

# ======================================================================
#  3. Preprocessing: lowercase -> tokenize -> stop-word removal -> stem
# ======================================================================
FALLBACK_STOPWORDS = {
    "a", "an", "and", "are", "as", "at", "be", "by", "for", "from", "how",
    "in", "is", "it", "of", "on", "or", "such", "that", "the", "their",
    "this", "to", "use", "using", "with",
}


def load_stopwords():
    try:
        nltk.download('stopwords', quiet=True)
        from nltk.corpus import stopwords
        return set(stopwords.words('english'))
    except Exception:
        return FALLBACK_STOPWORDS


STOP_WORDS = load_stopwords()
stemmer = PorterStemmer()


def preprocess(text):
    tokens = re.findall(r'[a-z]+', text.lower())
    return [stemmer.stem(t) for t in tokens if t not in STOP_WORDS]


class Index:
    """Holds term frequencies, document frequencies and lengths."""

    def __init__(self, docs):
        self.doc_ids = list(docs)
        self.N = len(docs)
        self.tf = {d: Counter(preprocess(title + " " + body))
                   for d, (title, body) in docs.items()}
        self.df = Counter()
        for counts in self.tf.values():
            self.df.update(counts.keys())
        self.length = {d: sum(c.values()) for d, c in self.tf.items()}
        self.avg_len = sum(self.length.values()) / self.N
```

```
    def idf(self, term):
        df = self.df.get(term, 0)
        return math.log10(self.N / df) if df else 0.0


# ======================================================================
#  4. Ranking Models
# ======================================================================
def log_tf(tf):
    return 1 + math.log10(tf) if tf > 0 else 0.0


def tfidf_score(index, query_terms):
    """TF-IDF: score(q, d) = sum over query terms of (1 + log tf) * idf."""
    scores = {}
    for d in index.doc_ids:
        scores[d] = sum(log_tf(index.tf[d][t]) * index.idf(t)
                        for t in set(query_terms))
    return scores


def vsm_score(index, query_terms):
    """Vector Space Model: cosine similarity between TF-IDF vectors (ltc.ltc)."""
    q_counts = Counter(query_terms)
    q_vec = {t: log_tf(c) * index.idf(t) for t, c in q_counts.items()}
    q_norm = math.sqrt(sum(w * w for w in q_vec.values()))
    scores = {}
    for d in index.doc_ids:
        d_vec = {t: log_tf(c) * index.idf(t) for t, c in index.tf[d].items()}
        d_norm = math.sqrt(sum(w * w for w in d_vec.values()))
        dot = sum(w * d_vec.get(t, 0.0) for t, w in q_vec.items())
        scores[d] = dot / (q_norm * d_norm) if q_norm and d_norm else 0.0
    return scores


def bim_term_weights(index, query_terms, relevant_set=None):
    """
    BIM term weight c_t = log[ p_t (1 - u_t) / (u_t (1 - p_t)) ]
      p_t = P(term present | relevant), u_t = P(term present | non-relevant).
    Without relevance information: p_t = 0.5, u_t ~ df_t / N.
    With a known relevant set V: p_t = (V_t + 0.5) / (|V| + 1),
                                 u_t = (df_t - V_t + 0.5) / (N - |V| + 1).
    """
    weights = {}
    for t in set(query_terms):
        df = index.df.get(t, 0)
        if relevant_set is None:
            p = 0.5
            u = (df + 0.5) / (index.N + 1)
        else:
            V = len(relevant_set)
            Vt = sum(1 for d in relevant_set if index.tf[d][t] > 0)
            p = (Vt + 0.5) / (V + 1)
            u = (df - Vt + 0.5) / (index.N - V + 1)
        weights[t] = math.log10((p * (1 - u)) / (u * (1 - p)))
    return weights


def bim_score(index, query_terms, relevant_set=None):
    """BIM Retrieval Status Value: sum of c_t over query terms present in d."""
    weights = bim_term_weights(index, query_terms, relevant_set)
    return {d: sum(w for t, w in weights.items() if index.tf[d][t] > 0)
            for d in index.doc_ids}


def bm25_score(index, query_terms, k1=1.5, b=0.75):
    """Probabilistic Retrieval Model (Okapi BM25): BIM + tf saturation + length norm."""
    scores = {}
    for d in index.doc_ids:
        s = 0.0
        for t in set(query_terms):
            tf = index.tf[d][t]
            if tf == 0:
                continue
            df = index.df[t]
            idf = math.log10((index.N - df + 0.5) / (df + 0.5) + 1)
            norm = k1 * ((1 - b) + b * index.length[d] / index.avg_len)
            s += idf * (tf * (k1 + 1)) / (tf + norm)
        scores[d] = s
    return scores
```

```python
MODELS = {
    "VSM (cosine)": vsm_score,
    "TF-IDF": tfidf_score,
    "BIM": bim_score,
    "BM25": bm25_score,
}


def rank(scores):
    """Sort by score (desc), break ties by document number; drop zero scores."""
    ranked = [(d, s) for d, s in scores.items() if s > 0]
    return sorted(ranked, key=lambda x: (-x[1], int(x[0][1:])))


# ============================================================================
#  5. Evaluation Metrics
# ============================================================================
def precision_at_k(ranking, rel, k):
    return sum(1 for d in ranking[:k] if rel.get(d, 0) > 0) / k


def recall_at_k(ranking, rel, k):
    total = sum(1 for g in rel.values() if g > 0)
    return sum(1 for d in ranking[:k] if rel.get(d, 0) > 0) / total if total else 0.0


def f_measure(p, r):
    return 2 * p * r / (p + r) if (p + r) else 0.0


def reciprocal_rank(ranking, rel):
    for i, d in enumerate(ranking, 1):
        if rel.get(d, 0) > 0:
            return 1 / i
    return 0.0


def average_precision(ranking, rel):
    total = sum(1 for g in rel.values() if g > 0)
    hits, s = 0, 0.0
    for i, d in enumerate(ranking, 1):
        if rel.get(d, 0) > 0:
            hits += 1
            s += hits / i
    return s / total if total else 0.0


def ndcg_at_k(ranking, rel, k):
    def dcg(grades):
        return sum((2 ** g - 1) / math.log2(i + 1) for i, g in enumerate(grades, 1))
    actual = dcg([rel.get(d, 0) for d in ranking[:k]])
    ideal = dcg(sorted(rel.values(), reverse=True)[:k])
    return actual / ideal if ideal else 0.0


def evaluate(ranking, rel, k):
    p = precision_at_k(ranking, rel, k)
    r = recall_at_k(ranking, rel, k)
    return {"P@5": p, "R@5": r, "F1@5": f_measure(p, r),
            "RR": reciprocal_rank(ranking, rel),
            "AP": average_precision(ranking, rel),
            "NDCG@5": ndcg_at_k(ranking, rel, k)}


# ============================================================================
#  6. Display helpers
# ============================================================================
def header(title):
    print("\n" + "=" * 75)
    print(f" {title}")
    print("=" * 75)


def show_ranking(name, ranking, rel, k):
    print(f"\n--- {name}: Top {k} ---")
    print(f"{'Rank':<5} {'Doc':<5} {'Score':>8}  {'Rel':<4} Title")
    print("-" * 75)
    for i, (d, s) in enumerate(ranking[:k], 1):
        mark = "Yes" if rel.get(d, 0) > 0 else "No"
        print(f"{i:<5} {d:<5} {s:>8.4f}  {mark:<4} {DOCUMENTS[d][0]}")
```

```python
def explain_prp(index, query_terms, rel):
    header("Probabilistic Ranking Principle (PRP)")
    print("""PRP (Robertson, 1977): if documents are ranked in decreasing order of
P(R = 1 | d, q), the expected effectiveness of the ranking is optimal
(under 1/0 loss and independent relevance). In this system:
  * BIM and BM25 follow PRP: they rank by log-odds of relevance, which is
    a monotonic function of P(R = 1 | d, q), so ordering is preserved.
  * BIM estimates term weights c_t = log[p_t(1-u_t) / (u_t(1-p_t))].
    With no relevance data p_t = 0.5 and u_t ~ df_t/N, so c_t ~ idf.
  * Once students mark some results relevant, p_t and u_t can be estimated
    from those judgments (relevance feedback). Better estimates of
    P(R | d, q) give a ranking closer to the PRP-optimal ordering.
  * VSM and TF-IDF are heuristic: their scores are similarities, not
    probabilities of relevance, so PRP does not justify them directly.""")

    relevant = sorted((d for d, g in rel.items() if g > 0), key=lambda d: int(d[1:]))
    w0 = bim_term_weights(index, query_terms)
    w1 = bim_term_weights(index, query_terms, relevant_set=relevant)
    print(f"\nBIM term weights c_t before and after relevance feedback "
          f"(V = {relevant}):")
    print(f"{'Term':<12} {'df':>4} {'V_t':>4} {'c_t (no feedback)':>20} "
          f"{'c_t (feedback)':>18}")
    print("-" * 63)
    for t in sorted(w0):
        vt = sum(1 for d in relevant if index.tf[d][t] > 0)
        print(f"{t:<12} {index.df.get(t, 0):>4} {vt:>4} {w0[t]:>20.4f} "
              f"{w1[t]:>18.4f}")

    # Re-rank with the feedback weights (illustration only: it uses the
    # judgments, so it is not part of the fair model comparison below)
    ranking = rank(bim_score(index, query_terms, relevant_set=relevant))
    show_ranking("BIM after relevance feedback (illustration)", ranking, rel, TOP_K)


# ======================================================================
#  7. Main
# ======================================================================
def main():
    output_dir = os.path.dirname(os.path.abspath(__file__))
    index = Index(DOCUMENTS)

    header("Tutorial 4: Search Engine Ranking and Evaluation (College Library)")
    print(f"Documents: {index.N}   Vocabulary size: {len(index.df)}   "
          f"Avg. doc length: {index.avg_len:.2f} terms")
    print("\n--- Document Collection ---")
    for d, (title, _) in DOCUMENTS.items():
        print(f"{d:<4} {title}")

    # ---------- Main query: ranked results ----------
    q_text = QUERIES[MAIN_QUERY]
    q_terms = preprocess(q_text)
    rel = JUDGMENTS[MAIN_QUERY]
    header(f"Ranked Results for {MAIN_QUERY}: \"{q_text}\"")
    print(f"Query terms after preprocessing: {q_terms}")
    print("\nIDF of query terms (log10 N/df):")
    for t in q_terms:
        print(f"  {t:<12} df = {index.df.get(t, 0):<3} idf = {index.idf(t):.4f}")

    main_results = {}
    for name, fn in MODELS.items():
        ranking = rank(fn(index, q_terms))
        main_results[name] = ranking
        show_ranking(name, ranking, rel, TOP_K)

    explain_prp(index, q_terms, rel)

    # ---------- Evaluation on the main query ----------
    header(f"Evaluation for {MAIN_QUERY} (relevant docs: "
           f"{sorted(d for d, g in rel.items() if g > 0)})")
    metric_names = ["P@5", "R@5", "F1@5", "RR", "AP", "NDCG@5"]
    print(f"{'Model':<14}" + "".join(f"{m:>9}" for m in metric_names))
    print("-" * 68)
    for name, ranking in main_results.items():
        res = evaluate([d for d, _ in ranking], rel, TOP_K)
        print(f"{name:<14}" + "".join(f"{res[m]:>9.4f}" for m in metric_names))

    # ---------- Evaluation across all queries ----------
    header("Evaluation over all test queries (MRR, MAP, mean NDCG@5)")
    for qid, text in QUERIES.items():
        print(f"{qid}: \"{text}\"  relevant = "
              f"{ {d: g for d, g in JUDGMENTS[qid].items()} }")
```

```
    summary = {}
    print(f"\n{'Model':<14}{'mean P@5':>10}{'mean R@5':>10}{'mean F1':>10}"
          f"{'MRR':>9}{'MAP':>9}{'NDCG@5':>9}")
    print("-" * 71)
    for name, fn in MODELS.items():
        per_query = [evaluate([d for d, _ in rank(fn(index, preprocess(text)))],
                              JUDGMENTS[qid], TOP_K)
                     for qid, text in QUERIES.items()]
        avg = {m: sum(r[m] for r in per_query) / len(per_query)
               for m in metric_names}
        summary[name] = avg
        print(f"{name:<14}{avg['P@5']:>10.4f}{avg['R@5']:>10.4f}"
              f"{avg['F1@5']:>10.4f}{avg['RR']:>9.4f}{avg['AP']:>9.4f}"
              f"{avg['NDCG@5']:>9.4f}")

    # ---------- Comparison plot ----------
    plot_metrics = ["P@5", "R@5", "F1@5", "RR", "AP", "NDCG@5"]
    labels = ["P@5", "R@5", "F1@5", "MRR", "MAP", "NDCG@5"]
    width = 0.2
    plt.figure(figsize=(11, 6))
    for i, name in enumerate(MODELS):
        xs = [j + (i - 1.5) * width for j in range(len(plot_metrics))]
        plt.bar(xs, [summary[name][m] for m in plot_metrics], width, label=name)
    plt.xticks(range(len(plot_metrics)), labels)
    plt.ylim(0, 1.05)
    plt.ylabel("Score (mean over queries)")
    plt.title("Ranking Model Comparison")
    plt.legend()
    plt.grid(axis='y', alpha=0.4)
    plot_path = os.path.join(output_dir, "model_comparison.png")
    plt.savefig(plot_path)
    plt.close()

    # ---------- Short report ----------
    best = max(summary, key=lambda m: (summary[m]["AP"], summary[m]["NDCG@5"]))
    header("Report: Which ranking model performs better?")
    for name in sorted(summary, key=lambda m: -summary[m]["AP"]):
        print(f"  {name:<14} MAP = {summary[name]['AP']:.4f}   "
              f"NDCG@5 = {summary[name]['NDCG@5']:.4f}")
    print(f"""
Best model on this test collection: {best}
Interpretation:
  * Precision@5 is the share of the top five results that students marked
    relevant; Recall@5 is the share of all relevant books found in the top
    five; F1 balances the two.
  * MRR shows how early the first useful book appears, MAP rewards placing
    all relevant books near the top, and NDCG@5 additionally rewards placing
    highly relevant (grade 2) books above partially relevant ones.
  * BIM ranks best here. Library records are short, so relevance depends
    on WHICH query concepts a book covers, not how often it repeats them.
    BIM scores only term presence, so books covering more distinct query
    terms (D1, D10, D4) rank higher.
  * VSM, TF-IDF and BM25 all use term frequency. D6 (a general machine
    learning textbook that repeats "machine learning" many times but never
    mentions healthcare) is pushed into their top five, which costs them a
    relevant book. BM25's tf saturation reduces this effect but does not
    remove it.
  * Among the tf-based models TF-IDF has the best MAP and NDCG. VSM and
    BM25 both normalise for document length. In a collection of short,
    similar-length records this adds little, and it slightly reorders
    relevant books lower down the ranking.
  * Every model ranks D9 (machine learning in finance) second for Q1
    because it matches 3 of the 4 query terms. Lexical models cannot tell
    that "healthcare" is the essential concept.
  * D11 (medical imaging) is missed by every model because it shares no
    query terms (vocabulary mismatch). Query expansion or semantic models
    would address this.
  * Caveat: with 20 documents and 3 queries, these differences are small.
    On large collections with longer documents, BM25 usually performs best.""")
    print(f"\nComparison plot saved to: {plot_path}")


if __name__ == '__main__':
    main()
```

Output

```
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials\Tutorial - 4> python main.py

================================================================================
 Tutorial 4: Search Engine Ranking and Evaluation (College Library)
================================================================================
Documents: 20   Vocabulary size: 177   Avg. doc length: 15.75 terms

--- Document Collection ---
D1   Machine Learning in Medical Diagnosis
D2   Healthcare Data Analysis using AI
D3   History of Computer Networks
D4   Deep Learning for Disease Prediction
D5   Database Management Systems
D6   Machine Learning: Theory and Algorithms
D7   Applications of Machine Learning in Healthcare
D8   Public Health and Healthcare Policy
D9   Machine Learning Applications in Finance and Marketing
D10  Natural Language Processing of Clinical Notes
D11  Computer Vision for Medical Imaging
D12  Operating System Concepts
D13  Reinforcement Learning for Robotics
D14  Organic Chemistry Fundamentals
D15  Thermodynamics for Engineers
D16  Principles of Microeconomics
D17  Data Structures and Algorithms
D18  Medieval Indian History
D19  Wearable Devices for Patient Monitoring
D20  Hospital Information Systems

================================================================================
 Ranked Results for Q1: "machine learning applications in healthcare"
================================================================================
Query terms after preprocessing: ['machin', 'learn', 'applic', 'healthcar']

IDF of query terms (log10 N/df):
  machin       df = 6   idf = 0.5229
  learn        df = 7   idf = 0.4559
  applic       df = 4   idf = 0.6990
  healthcar    df = 5   idf = 0.6021

--- VSM (cosine): Top 5 ---
Rank  Doc      Score  Rel  Title
--------------------------------------------------------------------------------
1     D7      0.3834  Yes  Applications of Machine Learning in Healthcare
2     D9      0.2719  No   Machine Learning Applications in Finance and Marketing
3     D1      0.1993  Yes  Machine Learning in Medical Diagnosis
4     D6      0.1776  No   Machine Learning: Theory and Algorithms
5     D10     0.1642  Yes  Natural Language Processing of Clinical Notes

--- TF-IDF: Top 5 ---
Rank  Doc      Score  Rel  Title
--------------------------------------------------------------------------------
1     D7      3.0722  Yes  Applications of Machine Learning in Healthcare
2     D9      2.1828  No   Machine Learning Applications in Finance and Marketing
3     D1      1.9676  Yes  Machine Learning in Medical Diagnosis
4     D6      1.5831  No   Machine Learning: Theory and Algorithms
5     D10     1.5809  Yes  Natural Language Processing of Clinical Notes

--- BIM: Top 5 ---
Rank  Doc      Score  Rel  Title
--------------------------------------------------------------------------------
1     D7      1.6180  Yes  Applications of Machine Learning in Healthcare
2     D9      1.1680  No   Machine Learning Applications in Finance and Marketing
3     D1      1.0537  Yes  Machine Learning in Medical Diagnosis
4     D10     1.0537  Yes  Natural Language Processing of Clinical Notes
5     D4      0.6037  Yes  Deep Learning for Disease Prediction

--- BM25: Top 5 ---
Rank  Doc      Score  Rel  Title
--------------------------------------------------------------------------------
1     D7      3.0452  Yes  Applications of Machine Learning in Healthcare
2     D9      2.2643  No   Machine Learning Applications in Finance and Marketing
3     D1      1.7997  Yes  Machine Learning in Medical Diagnosis
4     D6      1.6747  No   Machine Learning: Theory and Algorithms
5     D10     1.4076  Yes  Natural Language Processing of Clinical Notes

================================================================================
 Probabilistic Ranking Principle (PRP)
================================================================================
PRP (Robertson, 1977): if documents are ranked in decreasing order of
P(R = 1 | d, q), the expected effectiveness of the ranking is optimal
(under 1/0 loss and independent relevance). In this system:
  * BIM and BM25 follow PRP: they rank by log-odds of relevance, which is
    a monotonic function of P(R = 1 | d, q), so ordering is preserved.
  * BIM estimates term weights c_t = log[p_t(1-u_t) / (u_t(1-p_t))].
    With no relevance data p_t = 0.5 and u_t ~ df_t/N, so c_t ~ idf.
  * Once students mark some results relevant, p_t and u_t can be estimated
    from those judgments (relevance feedback). Better estimates of
    P(R | d, q) give a ranking closer to the PRP-optimal ordering.
  * VSM and TF-IDF are heuristic: their scores are similarities, not
    probabilities of relevance, so PRP does not justify them directly.

BIM term weights c_t before and after relevance feedback (V = ['D1', 'D2', 'D4', 'D7', 'D10', 'D11']):
Term           df  V_t    c_t (no feedback)     c_t (feedback)
----------------------------------------------------------------
applic          4    1               0.5643            -0.0476
healthcar       5    4               0.4500             1.2095
learn           7    4               0.2553             0.7719
machin          6    4               0.3485             0.9542

--- BIM after relevance feedback (illustration): Top 5 ---
Rank  Doc      Score  Rel  Title
--------------------------------------------------------------------------------
1     D1      2.9357  Yes  Machine Learning in Medical Diagnosis
2     D10     2.9357  Yes  Natural Language Processing of Clinical Notes
3     D7      2.8880  Yes  Applications of Machine Learning in Healthcare
4     D4      1.7261  Yes  Deep Learning for Disease Prediction
5     D6      1.7261  No   Machine Learning: Theory and Algorithms

================================================================================
 Evaluation for Q1 (relevant docs: ['D1', 'D10', 'D11', 'D2', 'D4', 'D7'])
================================================================================
Model               P@5      R@5     F1@5       RR       AP   NDCG@5
--------------------------------------------------------------------------------
VSM (cosine)     0.6000   0.5000   0.5455   1.0000   0.5722   0.7013
TF-IDF           0.6000   0.5000   0.5455   1.0000   0.5931   0.7013
BIM              0.8000   0.6667   0.7273   1.0000   0.6287   0.8614
BM25             0.6000   0.5000   0.5455   1.0000   0.5931   0.7013

================================================================================
 Evaluation over all test queries (MRR, MAP, mean NDCG@5)
================================================================================
Q1: "machine learning applications in healthcare"  relevant = {'D7': 2, 'D1': 2, 'D4': 2, 'D10': 2, 'D2': 1, 'D11': 1}
```

```
Q2: "deep learning for disease prediction"  relevant = {'D4': 2, 'D11': 1, 'D7': 1, 'D1': 1}
Q3: "database systems for hospital records"  relevant = {'D20': 2, 'D5': 2, 'D2': 1, 'D10': 1}

Model           mean P@5  mean R@5   mean F1      MRR      MAP   NDCG@5
------------------------------------------------------------------------
VSM (cosine)      0.6000    0.6667    0.6263   1.0000   0.7022   0.8257
TF-IDF            0.6000    0.6667    0.6263   1.0000   0.7615   0.8392
BIM               0.7333    0.8056    0.7609   1.0000   0.7924   0.9209
BM25              0.6000    0.6667    0.6263   1.0000   0.7129   0.8257

========================================================================
 Report: Which ranking model performs better?
========================================================================
  BIM            MAP = 0.7924   NDCG@5 = 0.9209
  TF-IDF         MAP = 0.7615   NDCG@5 = 0.8392
  BM25           MAP = 0.7129   NDCG@5 = 0.8257
  VSM (cosine)   MAP = 0.7022   NDCG@5 = 0.8257

Best model on this test collection: BIM
Interpretation:
  * Precision@5 is the share of the top five results that students marked
    relevant; Recall@5 is the share of all relevant books found in the top
    five; F1 balances the two.
  * MRR shows how early the first useful book appears, MAP rewards placing
    all relevant books near the top, and NDCG@5 additionally rewards placing
    highly relevant (grade 2) books above partially relevant ones.
  * BIM ranks best here. Library records are short, so relevance depends
    on WHICH query concepts a book covers, not how often it repeats them.
    BIM scores only term presence, so books covering more distinct query
    terms (D1, D10, D4) rank higher.
  * VSM, TF-IDF and BM25 all use term frequency. D6 (a general machine
    learning textbook that repeats "machine learning" many times but never
    mentions healthcare) is pushed into their top five, which costs them a
    relevant book. BM25's tf saturation reduces this effect but does not
    remove it.
  * Among the tf-based models TF-IDF has the best MAP and NDCG. VSM and
    BM25 both normalise for document length. In a collection of short,
    similar-length records this adds little, and it slightly reorders
    relevant books lower down the ranking.
  * Every model ranks D9 (machine learning in finance) second for Q1
    because it matches 3 of the 4 query terms. Lexical models cannot tell
    that "healthcare" is the essential concept.
  * D11 (medical imaging) is missed by every model because it shares no
    query terms (vocabulary mismatch). Query expansion or semantic models
    would address this.
  * Caveat: with 20 documents and 3 queries, these differences are small.
    On large collections with longer documents, BM25 usually performs best.

Comparison plot saved to: D:\SVNIT\Semester - 7\Information Retrieval\Tutorials\Tutorial - 4\model_comparison.png
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials\Tutorial - 4>
```